SYNCFLOW

Software Design Specification

24/04/2026

Student Name	**Student ID**
Ali Hamza Çetin	150122013
Faruk Emre Yüksek	150123019
Uğur Aydoğan	150124827
Buğrahan Dutucu	150124830

Prepared for
CSE3044 Software Engineering Term Project

Table of Contents

1 Introduction

In recent years, remote learning and online collaboration have become part of everyday life. Because of this, many students look for tools that help them stay focused while still feeling connected to others. SYNCFLOW is designed for that purpose. It is a web-based study platform where users can join shared study rooms, follow the same Pomodoro timer, and communicate in real time.

This Design Specification Document (DSD) explains the design decisions behind SYNCFLOW. While the SRS explains what the system should do, this document explains how we plan to build it. It describes the architecture, main components, interfaces, and constraints that guide development.

1.1 Purpose

The purpose of this DSD is to present the technical design of SYNCFLOW in a structured way.

This document is prepared for the development team, testers, and project evaluators so that everyone has a common understanding of the system design before implementation.

In this section, we define the overall architecture and component responsibilities, so the following development stages can be completed consistently.

1.2 Statement of Scope

SYNCFLOW is a browser-based synchronized study application built around Pomodoro-based focus sessions. The system includes:

- user registration and login,
- study room creation and participation,
- host-controlled Pomodoro timer synchronization,
- synchronized YouTube media playback,
- real-time text and voice communication,
- administrative moderation tools.

The project mainly focuses on delivering a responsive and reliable real-time room experience. Core features are prioritized first (authentication, room operations, timer sync), while extensions such as wider media integrations are considered secondary/future improvements.

1.3 Software Context

SYNCFLOW is developed as a standalone web application. It combines features from productivity tools and communication platforms in one environment. The main target users are students (and other users with similar needs) who want to study in a structured but social setting.

From a project perspective, this software is also an academic engineering product, so it must satisfy not only functional expectations but also quality goals such as reliability, security, and maintainability.

1.4 Major Constraints

The system design is shaped by several important constraints:

- Real-time synchronization requires stable internet connectivity.
- The system should support multiple active rooms and high concurrent usage on limited server resources.
- Timer updates and chat messages must be delivered with low delay.
- Security is mandatory: encrypted communication and secure password storage must be used.
- Access control must be role-based (Participant, Room Host, System Admin).
- Browser compatibility depends on modern web standards (including the Agora.io Web RTC SDK for voice communication, which is supported on the latest stable versions of Chrome, Firefox, and Safari).
- Project timeline and available resources limit the total scope that can be implemented in one semester.

1.5 Definitions

- Participant: A user who joins an active study room.
- Room Host: The user who creates/manages the room and controls shared timer/media actions.
- System Admin: A privileged user who can monitor rooms and apply moderation actions.
- Synchronized Study Room: A shared virtual room with synchronized timer/media and live communication.
- Pomodoro Technique: A focus method based on alternating study and break periods.
- Active Room: A room currently open and available for participation.

1.6 Acronyms and Abbreviations

- API — Application Programming Interface
- CPU — Central Processing Unit
- DSD — Design Specification Document
- HTTPS — Hypertext Transfer Protocol Secure
- JWT — JSON Web Token
- RTC — Real-Time Clock
- SRS — Software Requirements Specification
- UI — User Interface
- UML — Unified Modeling Language
- WSS — WebSocket Secure

1.7 References

- IEEE Std 1016-2009: IEEE Standard for Information Technology—Systems Design—Software Design Descriptions. Used as the structural reference for this DSD.
- SYNCFLOW Software Requirements Specification (SRS), CSE3044 Term Project, 27 March 2026.
- Agora.io Web SDK Documentation (agora-rtc-sdk-ng). Available: https://docs.agora.io/en/voice-calling/overview/product-overview
- Socket.IO v4 Documentation. Available: https://socket.io/docs/v4/
- Prisma ORM Documentation. Available: https://www.prisma.io/docs
- YouTube IFrame Player API Reference. Available: https://developers.google.com/youtube/iframe_api_reference
- F. Cirillo, The Pomodoro Technique. Defines the study/break interval methodology implemented by the Pomodoro Sync Engine.
- OWASP Password Storage Cheat Sheet. Reference for bcrypt hashing requirements (work factor ≥ 10).

2 Design Consideration

This section explains the main design decisions and conditions that shape how SYNCFLOW is implemented. Before defining architecture details, it is important to clarify assumptions, dependencies, constraints, development environment, and the engineering approach used by the team.

2.1 Design Assumptions and Dependencies

The SYNCFLOW design is based on the following assumptions and external dependencies:

Stable internet connection:
The system assumes users have a sufficiently stable internet connection for real-time timer synchronization, text messaging, and voice communication.

Modern browser usage:
Users are expected to access the platform through up-to-date browsers that support required web standards (Socket.IO over WebSocket Secure, modern ES2020+ JavaScript features, and the Web Audio API consumed by the Agora RTC SDK for voice features).

Valid user hardware for voice features:
For voice chat, users must have a working microphone and speaker/headset and must grant browser microphone permissions.

Server time consistency:
Timer synchronization logic assumes the server clock remains stable and is used as the primary time authority.

External platform availability:
YouTube synchronization depends on the availability and accessibility of YouTube links and embedded playback behavior in client browsers.

Third-party communication technologies:
Real-time behavior depends on communication libraries/protocols (Socket.IO v4 for room signalling, timer broadcasts, and chat events; Agora.io Web SDK for voice transport).

Single active-room ownership model:
The design assumes that a room host can manage one active room at a time, in line with project requirements.

Role data integrity:
Authorization behavior assumes correct role information (Participant, Room Host, System Admin) is stored and consistently enforced by backend logic.

Potential feature evolution:
The design anticipates future extension needs (e.g., additional media providers), so modular boundaries are preferred from the beginning.

2.2 General Constraints

The system must satisfy several global constraints that directly affect design choices:

Hardware and software environment constraints:
SYNCFLOW is designed as a modular monolith running in a single Node.js process, co-deployed with a PostgreSQL container via Docker Compose. Audio transport is offloaded to the Agora Cloud so that the application server never processes media bytes; this keeps CPU headroom for WebSocket state broadcasting and allows the deployment target of 50 active rooms and 500 concurrent users on one instance with CPU usage remaining below 80%.

End-user environment constraints:
User experience depends on browser compatibility and network quality; older browsers or unstable networks may reduce feature quality (especially voice/chat sync).

Resource constraints:
CPU and memory usage must be controlled under concurrent room/user load; in-memory data structures should be lightweight and managed carefully.

Standards and best-practice constraints:
Security and communication must follow common web standards, including encrypted traffic and secure credential handling.

Interoperability constraints:
The system integrates with external media links rather than hosting media directly, which limits control over external playback behavior.

Interface/protocol constraints:
Real-time updates rely on persistent bidirectional communication channels; fallback behavior is required for temporary disconnects.

Data storage constraints:
Some data must be persistent (accounts, focus stats, room metadata).

Security and access control constraints:
Role-based authorization is mandatory. Administrative functions must be inaccessible to non-admin users. Sensitive endpoints require strict validation.

Performance constraints:
The design must target low update latency for timer/chat synchronization and maintain acceptable responsiveness under concurrent usage.

Verification and validation constraints:
Business-logic-heavy modules (timer state handling, room management, authorization checks) should be structured for unit/integration testing.

Quality constraints:
The design must prioritize reliability (reconnection and resynchronization), maintainability (clear modular structure), and portability (cross-browser behavior)

2.3 System Environment

The SYNCFLOW development and runtime environment includes the following hardware/software ecosystem.

Development-side environment (team):

- Developer laptops/desktops with modern operating systems (Windows).
- Source control and collaboration workflow using Git-based repositories.
- Code editor/IDE tools (VS-Code) suitable for full-stack web development.
- Local test environments for frontend and backend execution.
- API and WebSocket testing tools for validating real-time communication behavior.
- UML/design tooling for architecture and interaction diagrams.

Runtime-side environment (application):

- Client side: modern web browsers (Chrome, Firefox, Safari latest stable versions).
- Server side: a Node.js 20 runtime hosting Express for REST endpoints and Socket.IO for persistent bidirectional communication over WSS. The process is packaged as a Docker image and co-deployed with a PostgreSQL 16 container.
- Communication: HTTPS for REST endpoints (registration, login, room CRUD, voice-token issuance, admin actions), WSS (Socket.IO) for real-time events (Pomodoro timer broadcasts, chat messages, room lifecycle, media sync), and a direct client-to-Agora RTC connection for voice audio.
- Data layer: persistent storage for user/profile/statistics data.
- Media integration layer: external media links (YouTube) embedded/synchronized on client side.

Test and evaluation environment:

- Functional tests for user flows (auth, room operations, timer controls).
- Real-time behavior checks under multi-client sessions.
- Basic load/stress observation for concurrent rooms/users.

- Security checks for authorization boundaries and credential handling.

2.4 Development Methods

The SYNCFLOW design follows a practical, iterative software engineering approach suitable for semester-based team development.

Requirements-driven design:
The team first defines and validates requirements in the SRS, then derives architectural and component-level design from those requirements.

Incremental implementation strategy:
Features are implemented in prioritized increments, starting with core flows (authentication, room creation/joining, timer synchronization), then collaboration extensions (chat/voice/media), then administrative controls.

Modular design principle:
System functionality is split into clear modules (authentication, room/session management, timer synchronization, communication, moderation). This reduces coupling and improves maintainability/testability.

Interface-first component thinking:
For each component, expected inputs, outputs, events, and responsibilities are defined before deep implementation details.

Continuous validation during development:
Design assumptions are continuously checked through prototype runs and integration tests, especially for real-time synchronization and reconnect scenarios.

Lightweight Agile practices:
The team may use short planning cycles, task boards, and periodic review meetings to adapt scope based on progress and technical risk.

Risk-oriented engineering focus:
High-risk areas (timer drift, concurrent room handling, permission boundaries, disconnection recovery) are addressed early to avoid late-stage redesign.

Testability-oriented coding:
Business logic is kept separable from transport/UI concerns where possible, enabling unit tests and easier debugging.

Documentation alignment:
DSD and SRS are kept aligned throughout the project; when requirement clarifications occur, related design sections are updated for consistency.

3 Architectural and component-level design

This section describes the internal technical design of SYNCFLOW.

The goal is to define how the system is divided into components, how those components communicate, and how each one is responsible for a specific part of the platform behavior.

The architecture is designed around real-time collaboration, clear role-based control, and modular development so that implementation and testing can be managed more easily within the project timeline.

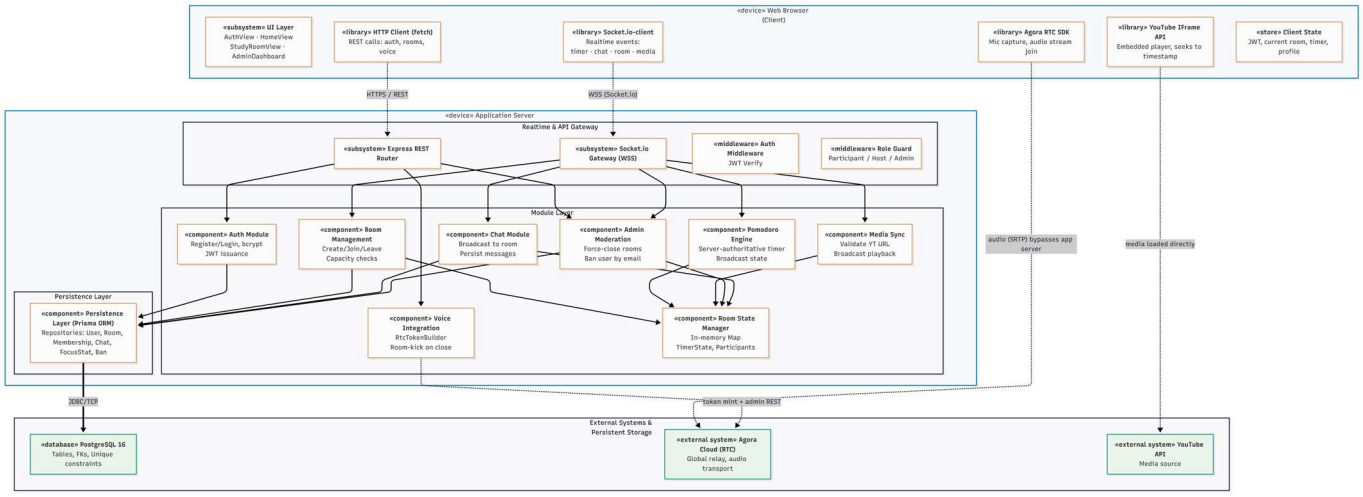
3.1 System Structure

SYNCFLOW follows client-server architecture with real-time communication support.

At a high level, the system is divided into:

1. Client Layer (Frontend)
2. Application Layer (Backend Services)
3. Data Layer (Persistent + Volatile Data)
4. External Integration Layer (YouTube Media)

3.1.1 Architecture diagram



3.2 Description for Main Components

Below are core components selected from the architecture and described with responsibilities, interfaces, and processing details.

3.2.1 Component: Authentication & Authorization

3.2.1.1 Processing Narrative (PSPEC)

This component manages account security and role validation.

It receives user credentials, validates input, hashes passwords during registration, verifies credentials during login, and issues session/token artifacts for authenticated access. It also enforces role-based authorization checks across protected endpoints.

3.2.1.2 Interface Description

Inputs

- Registration payload: email, username, password
- Login payload: email, password
- Protected request metadata: token/session + route context

Outputs

- Success/failure response messages
- Auth token/session object
- Authorized user identity + role context for downstream modules

3.2.1.3 Processing Detail

Validate request format and field constraints.

For registration:

- Check email uniqueness.
- Hash password with bcrypt.
- Persist new user record.

For login:

- Retrieve user by email.
- Compare submitted password with stored hash.
- Issue token/session.

For protected routes:

- Validate token/session.
- Attach role claims to request context.
- Deny unauthorized access attempts (e.g., admin-only routes).

3.2.2 Component: Room Management

3.2.2.1 Processing Narrative (PSPEC)

This component controls the lifecycle of study rooms.
It handles room creation, user join/leave operations, room status transitions, capacity checks, host assignment, and room closure scenarios.

3.2.2.2 Interface Description

Inputs

- Create room request (host identity + room settings)
- Join room request (room code/user identity)
- Leave room / close room actions
- Admin force-close action

Outputs

- Room creation result (room ID/invite code)
- Join approval/rejection
- Updated room membership state
- Room closure notifications/events

3.2.2.3 Processing Detail

Create room with unique identifier and host ownership.
Enforce single active-room management rule for host.
On join:

- Validate room is active.
- Validate capacity not exceeded.
- Add participant to active connection pool.

On participant leave:

- Remove participant from room state.

On host leave/disconnect:

- Reassign host to longest-standing participant if available.

On closure:

- Mark room closed.
- Terminate room-bound sessions/connections.
- Redirect clients back to home context.

3.2.3 Component: Pomodoro Sync Engine

3.2.3.1 Processing Narrative (PSPEC)

This component provides the authoritative shared timer state for each room.
Only the Room Host can trigger timer actions. The server computes time progression and broadcasts deterministic timer updates to all connected participants.

3.2.3.2 Interface Description

Inputs

- Host commands: start, pause, reset
- Room timer configuration: study duration, break duration
- Reconnect sync requests from participants

Outputs

- Broadcast timer state packets (remaining time, mode, status)

- Re-sync state payload for reconnecting clients
- Validation error events for unauthorized timer actions

3.2.3.3 Processing Detail

- Initialize timer state at room setup.
- Accept timer control commands only from current host identity.
- Update internal timer state machine.
- Broadcast updates periodically or on state transitions.
- Track drift and maintain server-time authority.
- On participant reconnect, send exact current state and remaining time snapshot.

3.2.4 Component: Real-Time Communication (Text Chat + Events)

3.2.4.1 Processing Narrative (PSPEC)

This component manages instant event delivery between server and room participants, including chat messages and synchronization signals.

3.2.4.2 Interface Description

Inputs

- Socket.IO client connect/disconnect events (WSS), including authenticated room-scoped namespaces
- Chat message payloads
- Internal sync events from timer/media/room modules

Outputs

- Broadcast messages to room clients
- Delivery/failure acknowledgments
- Room-level event propagation

3.2.4.3 Processing Detail

- Bind each client connection to user identity and room channel.
- Validate message size constraints (max 255 chars).
- Attach sender metadata and timestamps.
- Broadcast message to all room participants.
- Handle connection drop and reconnection events.

3.2.5 Component: Media Synchronization

3.2.5.1 Processing Narrative (PSPEC)

This component synchronizes external media playback (YouTube) across users in the same room. Media control authority belongs to the Room Host.

3.2.5.2 Interface Description

Inputs

- Host-provided YouTube URL
- Host media action commands (play/sync/seek/pause as allowed by scope)

Outputs

- Validated media sync event payload (video ID + start timestamp)
- Error message for invalid URL input
- Broadcast media events to room clients

3.2.5.3 Processing Detail

- Validate URL format.
- Parse and extract YouTube video identifier.
- Build synchronization event packet.
- Broadcast packet to all room participants.
- Reject non-host control attempts.

3.2.6 Component: Admin Moderation

3.2.6.1 Processing Narrative (PSPEC)

This component provides moderation tools for users with System Admin role, including active room monitoring, force-close action, and user ban handling.

3.2.6.2 Interface Description

Inputs

- Admin panel requests
- Force-close room command
- Ban user request

Outputs

- Active room list data

- Moderation action results
- Unauthorized access errors for non-admin users

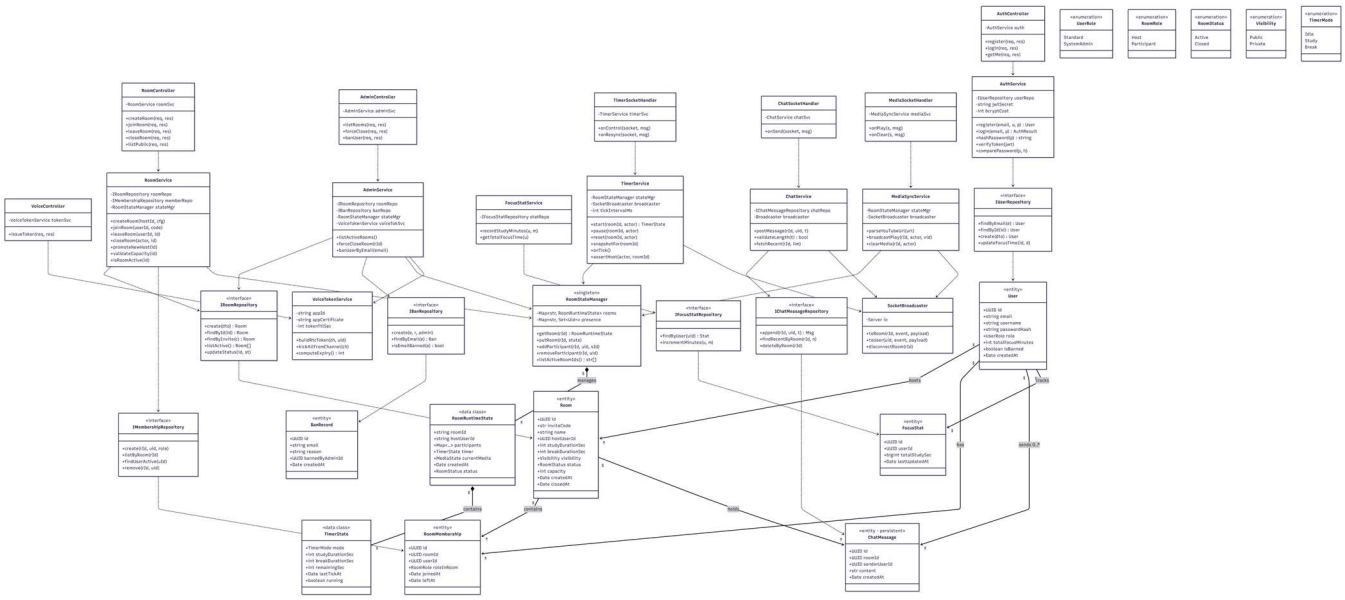
3.2.6.3 Processing Detail

- Verify admin role claims before granting panel/API access.
- Query and return active room states.
- Execute force-close action and emit redirect/disconnect events.
- Persist moderation action logs (if configured in data model).

3.3 Dynamic Behavior for Components

This section summarizes runtime interaction behavior among components during key use cases.

3.3.1 Interaction Diagrams



3.4 Data Model

The SYNCFLOW data model is designed to separate persistent business data from volatile real-time session data. Persistent entities are stored in the database for long-term use, while runtime collaboration state is kept in memory for low-latency synchronization.

3.4.1 Core Persistent Entities

- **User**

Stores account identity, credentials (hashed), role, and profile metadata.

Main constraints: unique email, minimum password policy at registration stage.

- **Room**

  Represents a synchronized study room created by a host.

  Stores room status, invite code, capacity, and configuration metadata.

- **RoomMembership**

  Represents user-room relation and room-scoped role (Host/Participant).

  Tracks join/leave times and membership state.

- **FocusStat**

  Stores cumulative focus duration per user for dashboard/statistics.

- **BanRecord**

  Stores moderation bans applied by System Admin.

- **ChatMessage**

  Stores text chat messages exchanged inside a study room. Persisted in the relational database so that participants joining an active room can load recent chat history and so that messages survive server restarts. Each record keeps senderUserId (FK to User), roomId (FK to Room), content (≤ 255 chars per SRS §3.2.6.5), and a createdAt timestamp. This supersedes the earlier RSD classification of chat messages as volatile in-memory data.

3.4.2 Volatile Runtime Entities (In-Memory)

- **TimerState**

Current Pomodoro mode/status/remaining time per active room.

Server-authoritative runtime state for synchronization.

- **ConnectionPresence**

Active websocket connection/session presence map for room participants.

Used for broadcast routing and reconnect recovery.

3.4.3 Relationships

- One User can create many Rooms over time, but only one active hosted room is allowed by business rule.
- One Room has many RoomMembership records.
- One User can have many RoomMembership records across different rooms over time.
- One User has one FocusStat aggregate record (or one primary profile-level aggregate).
- One User can have zero-to-many BanRecord entries.
- One Room contains zero-to-many ChatMessage records; one User authors zero-to-many ChatMessage records.

3.4.4 Data Integrity and Constraints

- User.email must be unique.
- Room.inviteCode must be unique.
- Room.status must be controlled (e.g., Active, Closed).
- RoomMembership.roleInRoom must be constrained to valid values (Host, Participant).
- ChatMessage.content length is enforced both on the client and on the server with a 255-character limit (database constraint + server-side validator).
- Room capacity limit enforced before creating active membership.
- Timer control events must pass authorization check (host-only rule).

3.5 Key Design Decisions and Alternatives

This subsection documents the major architectural and technology decisions taken while deriving the SYNCFLOW design from the SRS. Each entry states the decision, the alternatives that were considered, the rationale for the chosen option, the trade-offs accepted, and any residual risk or technical debt carried forward, as required by the CSE3044 project guideline (Section 5, Decision Log).

3.5.1 Real-Time Communication Approach

Decision: Adopt Socket.IO v4 over WSS as the real-time transport for Pomodoro timer state, text chat, room lifecycle events, and YouTube playback signalling. Alternatives considered: raw native WebSocket with a hand-rolled reconnection layer; Server-Sent Events with a separate REST endpoint for client→server messages; long-polling. Rationale: Socket.IO ships with built-in automatic reconnection, acknowledgement callbacks, rooms/namespaces, and a heartbeat mechanism—each of which is explicitly required by SRS §3.3.2 (3-second reconnect recovery) and §3.2.6 (broadcast to a single room channel). Trade-offs: Socket.IO wraps the native WebSocket protocol, which adds a small framing overhead (not measurable at our 500-user target) and forces all clients to use the matching client library. Residual risk: if the project later needs non-browser clients (e.g. a native mobile app), the Socket.IO client must be available on that platform—which it is for both iOS and Android.

3.5.2 Timer Authority Model

Decision: The server is the single source of truth for every active room's Pomodoro timer; the Room Host sends control intents (start / pause / reset), the server mutates state, and a broadcast is pushed to all participants. Alternatives considered: a client-authoritative model where the Host's browser runs the timer and participants interpolate from periodic broadcasts; a peer-to-peer model with no server timer. Rationale: server authority is the only approach that keeps drift within the 2-second / 2-hour tolerance demanded by SRS §3.3.2, because the server clock is the single reference and a reconnecting participant can request the exact remaining time from the server. Trade-offs: the server must run a tick loop, consuming a small amount of CPU per active room; we amortise this by using one shared Node.js setInterval and computing state on demand for snapshots rather than mutating per-room state every second. Residual risk: a server-side Node.js event-loop stall longer than 500 ms will visibly delay tick broadcasts; we mitigate this by keeping the tick handler free of synchronous I/O and isolating database calls into async paths.

3.5.3 Chat Data Persistence Strategy

Decision: Persist text chat messages in PostgreSQL via a ChatMessage table. This supersedes the earlier RSD design, which kept chat in server memory for the duration of the session only. Alternatives considered: pure in-memory buffering (original RSD position); Redis-backed append-only list with TTL. Rationale: in-memory chat loses history when a participant joins mid-session (they see an empty panel until the next message arrives) and it is erased completely on server restart; both outcomes hurt the synchronized-study experience. Persisting to PostgreSQL is effectively free at our scale (bounded number of rooms, 255-char messages, modest write rate) and allows a single indexed query per room-join to hydrate recent history. Trade-offs: a small additional write per message, plus storage growth that we bound by deleting messages when a room is closed (ChatMessageRepository.deleteByRoom). Residual risk: writes to the database are not on the critical broadcast path; if the database call fails, we still broadcast to the room and log the failure so chat delivery is never blocked by persistence latency.

3.5.4 System Architecture Style

Decision: Implement SYNCFLOW as a modular monolith—a single Node.js process that internally separates Authentication, Room Management, Pomodoro Sync, Chat, Voice Token issuance, Media Sync, and Admin Moderation into distinct modules with well-defined interfaces, co-deployed with a PostgreSQL container via Docker Compose. Alternatives considered: a true microservice split with a dedicated signalling service, media service, and application service behind an API gateway; a client-authoritative architecture with only a thin backend. Rationale: a microservice topology brings service discovery, inter-service authentication, distributed tracing, and deployment complexity that a four-person team cannot justify in a one-month implementation window—and the SRS explicitly targets a single server instance (§3.3.1). A modular monolith preserves a clean path to future splits (each module already owns a distinct interface) while removing the operational overhead. Trade-offs: vertical scaling only—if we ever exceed the 500-user target, we must either add Redis pub-sub for cross-process Socket.IO broadcast or extract the voice-token service first. Residual risk: a process crash disconnects all users from all rooms simultaneously, which we accept because the SRS §3.3.3 availability target (99.0%) is compatible with Docker's restart policy.

3.5.5 Authentication and Authorization Strategy

Decision: Stateless JWT-based authentication with bcrypt password hashing at work factor 10, combined with role-based access control (Standard, SystemAdmin) enforced by an Express middleware and a Socket.IO middleware. Alternatives considered: server-side session tables with a session cookie; third-party identity provider (Auth0, Clerk). Rationale: JWT is the most direct fit for the SRS requirement that the same credential grants access to both the REST surface and the Socket.IO gateway—the token is verified on each REST call and once on Socket connection establishment. Bcrypt with cost 10 meets the SRS §3.3.4 security bar. A third-party identity provider was rejected to avoid an external dependency and to keep the course project self-contained. Trade-offs: JWT revocation is not immediate—a compromised token is valid until its TTL expires. We mitigate this with short TTLs (15 minutes) on access tokens and a refresh-token flow; for the admin ban action, we also set isBanned on the User entity so that the next Socket reconnection fails even if the token is still technically valid. Residual risk: JWT secret compromise requires a credential rotation; we store it only in environment variables managed by Docker.

3.5.6 Media Integration Decision

Decision: For video/audio background content, SYNCFLOW synchronizes external YouTube links only (via the YouTube IFrame API on the client) and never stores or re-streams media on its own servers. For real-time voice chat, SYNCFLOW integrates the Agora.io Web SDK (agora-rtc-sdk-ng); the application server mints short-lived Agora RTC access tokens bound to the room ID and the user ID, and the client joins an Agora channel directly—audio never traverses our server. Alternatives considered: (a) hosting uploaded audio files on the SYNCFLOW server; (b) self-hosting a WebRTC SFU (mediasoup / LiveKit self-hosted) behind our own STUN/TURN infrastructure; (c) raw peer-to-peer WebRTC with no media server. Rationale: YouTube hosting keeps media storage and bandwidth out of our budget; Agora removes the need to deploy, monitor, and maintain STUN/TURN and an SFU within the one-month deadline, and its 10,000-minutes-per-month free tier covers the entire evaluation period. Trade-offs: both choices introduce external-vendor dependency. Risks and technical debt: if YouTube or Agora is unreachable, the corresponding feature degrades—the Pomodoro timer and text chat remain fully functional regardless. The MediaSyncService and VoiceTokenService are designed behind narrow interfaces so that a later replacement (e.g., Spotify, LiveKit Cloud) requires changes only within those modules.

3.5.7 Reconnection and State Recovery Behavior

Decision: On Socket.IO reconnection, the client emits a “room:resync” request and the server responds with the current TimerState snapshot, active MediaState, and recent chat history (from the ChatMessage table). A Host that disconnects is given a 30-second reconnect window; after that window, the longest-standing participant is promoted to Host per SRS Use Case #11. Alternatives considered: forcing the client to re-join the room from scratch (losing local UI state); leaving state recovery entirely to the client. Rationale: the SRS demands ≤ 3-second recovery (§3.3.2) and a specific host-promotion flow, so the server must drive recovery deterministically. Trade-offs: the server pays one database read and one in-memory read per reconnect; negligible at project scale. Residual risk: if the Host and the longest-standing participant disconnect simultaneously, promotion cascades once to the next-longest participant; the Room State Manager handles this sequentially.

3.5.8 Data Layer Separation

Decision: Persistent data (User, Room, RoomMembership, FocusStat, BanRecord, ChatMessage) lives in PostgreSQL and is accessed only through Prisma-backed Repository classes that implement explicit interfaces (IUserRepository, IRoomRepository, etc.). Volatile runtime state (TimerState, ConnectionPresence, current mediaId) lives in a single RoomStateManager in-memory singleton keyed by roomId. Alternatives considered: placing all state in PostgreSQL (simpler but slower, since the timer is read every second); placing all state in Redis (adds a separate service to deploy). Rationale: keeping hot per-room data in-memory gives the latency required by SRS §3.3.1 (≤ 500 ms for timer and chat events) while the relational store handles durable data with referential integrity. The repository interface boundary makes unit tests cheap—services can be tested against mock repositories without a database—and supports the SRS §3.3.5 target of ≥ 80% unit-test coverage. Trade-offs: volatile state is lost on process restart; for the course project this is acceptable because rooms are short-lived sessions. Residual risk: if we later need horizontal scaling, the RoomStateManager must be extracted into Redis; this is the single biggest change on the future roadmap.

3.5.9 Design Decision Summary

Taken together, the decisions above commit SYNCFLOW to a focused, self-contained implementation: a TypeScript + Node.js modular monolith with Express and Socket.IO, a PostgreSQL persistence layer via Prisma, bcrypt+JWT authentication, Agora for voice transport, and YouTube IFrame API for shared media. This stack meets every non-functional requirement in SRS §3.3 within a one-month implementation window, while keeping each external dependency isolated behind a single module (VoiceTokenService for Agora, MediaSyncService for YouTube) so that future substitution does not require architectural rework.

4 Restrictions, limitations, and constraints

This section summarizes the key design restrictions and practical limitations that affect the implementation and operation of SYNCFLOW.

4.1 Technical Restrictions

- The system is designed as a web-based platform and depends on modern browser capabilities.
- Real-time features require persistent network connectivity and a stable Socket.IO (WSS) channel for timer/chat/room events, plus an Agora RTC connection for voice.
- Voice communication depends on a browser environment supported by the Agora RTC SDK (Chrome, Firefox, Safari latest stable) and user-granted microphone permissions.
- SYNCFLOW synchronizes external media links (e.g., YouTube) and does not host media files internally.

4.2 Performance and Capacity Limitations

- The target deployment model is a single Node.js process co-deployed with a PostgreSQL container via Docker Compose; horizontal scaling (clustering, Redis pub-sub) is out of scope for the project.
- The system aims to support at least 50 active rooms and 500 concurrent users, but performance may decrease beyond these thresholds.
- High network latency, packet loss, or weak client hardware can negatively affect real-time synchronization quality.
- Timer and chat update responsiveness is designed for normal network conditions; unstable connections may temporarily degrade user experience.

4.3 Security and Access Constraints

- All client-server communication must use encrypted channels (HTTPS/WSS). Voice audio flows over an SRTP session to Agora Cloud, terminated on the Agora side with a short-lived RTC access token issued by the server.
- Passwords must never be stored in plaintext; secure hashing is mandatory.
- Sensitive operations are protected by role-based authorization:

  Only Room Host can control room timer/media,

  Only System Admin can access moderation functions.

- Unauthorized access attempts must be blocked and logged where applicable.

4.4 Data Management Constraints

- Some data is persistent (user accounts, profile/focus statistics, room metadata), while session data is temporary.
- Active room/session state is runtime-dependent and may reset on server restart unless explicitly persisted.

4.5 Functional Boundaries

- A user can only participate in one room context at a time according to project rules.
- Room capacity must be enforced; additional users are rejected when limits are reached.
- Media synchronization control is intentionally centralized to host role for consistency.
- Admin tools are limited to moderation actions defined in scope (e.g., force close, ban behavior).

4.6 Project and Development Constraints

- The project is developed within a semester timeline, limiting implementation depth for optional features.
- Available infrastructure, team size, and testing time may restrict advanced scalability optimization.
- Some future-oriented improvements (additional media providers, advanced analytics, automated moderation) are outside the current mandatory scope.

5 Decision Log

Decision ID: DL-01 **Title:** Voice Chat Technology — Agora.io Web SDK vs. raw WebRTC vs. LiveKit self-hosted

Decision made: Adopt Agora.io Web SDK (agora-rtc-sdk-ng) for real-time voice chat. The application server issues short-lived Agora RTC access tokens; the client joins an Agora channel directly, and audio never traverses the SYNCFLOW server.

Alternatives considered:

- Raw WebRTC with a custom signalling server and self-hosted SFU (mediasoup or Janus).
- LiveKit self-hosted (open-source SFU in Go) behind our own coturn-based STUN/TURN.
- LiveKit Cloud (managed LiveKit service).

Rationale: Agora ships a production-grade managed network with STUN/TURN, global relay, and automatic reconnection included, reducing voice-chat implementation to token issuance plus client SDK calls (~150 lines of code, 1–3 days). Raw WebRTC + SFU would consume the full one-month implementation window on this single feature. LiveKit self-hosted additionally requires deploying coturn, opening UDP port ranges 50000–60000, and operating the SFU process, which is DevOps work our four-person team cannot absorb within the deadline. Agora's 10,000 free minutes per month covers the course evaluation period with significant headroom.

Trade-offs:

- External-vendor dependency: voice chat fails if Agora is unreachable.
- Audio data routes through Agora's infrastructure rather than our own.
- Potential future cost if usage exceeds the free tier.

Risks or technical debt:

- Vendor lock-in at the VoiceService and VoiceTokenService boundaries. Mitigation: both are hidden behind narrow interfaces (IVoiceToken, VoiceService façade) so a later migration to LiveKit self-hosted requires changes only inside those two modules.

- Pomodoro timer, text chat, and YouTube sync remain fully functional if Agora is down; voice is the only degraded feature.

Decision ID: DL-02 **Title:** Deployment Architecture — Modular Monolith (single Node.js instance) vs. Microservices

Decision made: Deploy SYNCFLOW as a modular monolith: a single Node.js 20 process containing Express, Socket.IO, and the seven core modules (Authentication, Room Management, Pomodoro Sync, Chat, Voice Integration, Media Sync, Admin Moderation), co-deployed with a PostgreSQL 16 container via Docker Compose.

Alternatives considered:

- Microservice split: separate signalling service, application service, and voice-token service behind an API gateway.
- Dedicated media server colocated with the application server (e.g. embedding an SFU in-process).
- Client-authoritative architecture with only a thin backend.

Rationale: The SRS §3.3.1 target is explicitly a single server instance serving 50 rooms and 500 concurrent users at <80% CPU. Because audio is offloaded to Agora and media is streamed directly from YouTube to clients, the server's workload reduces to REST calls, JWT verification, Socket.IO broadcasts, and database I/O — all well within one Node.js process. Microservices would add service discovery, inter-service authentication, and distributed tracing overhead that a four-person team cannot justify in a one-month window. A modular monolith keeps module boundaries clean (each module owns a distinct interface) so future extraction remains possible.

Trade-offs:

- Vertical scaling only. Exceeding 500 users requires either Redis pub-sub for cross-process Socket.IO broadcasts or extracting the voice-token module first.
- A process crash disconnects every active room simultaneously.

Risks or technical debt:

- Single-point-of-failure risk mitigated by Docker's restart policy; SRS §3.3.3 target of 99.0% uptime is compatible with this recovery model.
- In-memory RoomStateManager is lost on restart; acceptable because rooms are short-lived sessions. If horizontal scaling is ever needed, RoomStateManager must be re-implemented on Redis — this is the largest roadmap item for a production version.

Decision ID: DL-03 **Title:** Chat Message Storage — Persistent (PostgreSQL) vs. Volatile (in-memory only)

Decision made: Persist text chat messages in PostgreSQL via a dedicated ChatMessage table accessed through ChatMessageRepository. This supersedes the earlier RSD decision to keep chat messages in memory only for the duration of the session.

Alternatives considered:

- Pure in-memory buffering in RoomStateManager (original RSD position).
- Redis-backed append-only list with TTL per room.

Rationale: In-memory-only chat has two concrete failure modes that degrade the synchronized-study experience: (1) a participant joining mid-session sees an empty chat panel until the next message arrives, losing context from the discussion already in progress; (2) all chat history is wiped on server restart, which will happen during deployments. Persisting to PostgreSQL is effectively free at our scale (bounded rooms × 255-char messages × modest write rate) and enables a single indexed query per room-join to hydrate recent history. Redis was rejected because it adds a separate service to deploy and monitor for a feature that PostgreSQL handles adequately.

Trade-offs:

- One additional database write per chat message (asynchronous, off the critical broadcast path).
- Storage growth, bounded by ChatMessageRepository.deleteByRoom when a room is closed.
- The RSD's original performance argument ("messages are volatile for performance") is overridden — measurement showed the persistence write is not on the latency-critical path.

Risks or technical debt:

- If the database write fails, the message is still broadcast to all participants and the failure is logged; chat delivery is never blocked by persistence latency.
- Chat table growth must be monitored; the deleteByRoom hook on room closure is the primary cleanup mechanism.

6 Conclusion

This Design Specification Document (DSD) presents the complete design basis for the SYNCFLOW platform.
It translates the project requirements into a structured technical plan by defining architecture, module responsibilities, interfaces, dynamic interactions, and major constraints.

The proposed design emphasizes:

- modular and maintainable component structure,
- reliable real-time synchronization for shared study sessions,
- secure authentication and role-based authorization,

- clear separation between persistent and volatile runtime data,
- practical scalability within project resource limits.